VIETTEL DIGITAL TALENT 2026

Streaming Processing in BigData Platform

Real-time data pipelines with Apache Kafka & Apache Flink

Mentor: Nguyen Minh Hieu
Student: Ngo Quang Hieu

Data Engineer Track
Viettel Digital Talent 2026

July 1^{st}, 2026

Table of Contents

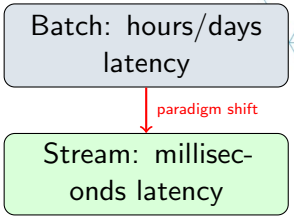
Table of Contents

Motivation

- Global data generation exceeds **2.5 quintillion bytes/day**
- Most of this value is **time-sensitive** — stale data = lost opportunity
- Traditional **batch pipelines** (Hadoop, scheduled SQL): hourly/daily latency
- A music platform cannot recommend based on what a user played *an hour ago*

Stream processing enables:

- Real-time personalisation in **milliseconds**
- Operational dashboards reflecting **current** state
- Event-driven microservices & fault-tolerant pipelines



Problem Statement

A music streaming platform generates **3 real-time event types**:

listen_events	**page_view_events**	**auth_events**
Artist, song, duration, user location, session	Navigation, HTTP method, status code, user context	Login/logout, session validity, subscription level

Why batch is insufficient — the 4 Vs:

- **Velocity** — thousands of events/second; buffering for hours wastes time-sensitivity
- **Volume** — accumulated data grows rapidly; batch windows grow ever larger
- **Variety** — 3 schemas, different transforms, must join dimension data (users, songs)
- **Reliability** — outages must not cause data loss; resume from last committed offset

Core challenge: design a fault-tolerant, low-latency pipeline that ingests heterogeneous events, transforms them in real time, and delivers clean data to a cloud warehouse — with exactly-once guarantees.

Objectives & Scope

Objectives:

1. Survey stream-processing concepts (watermarks, stateful computation, delivery semantics)
2. Evaluate **Apache Kafka** as the event transport layer
3. Compare **Apache Spark vs Apache Flink**
4. Build **Streamify**: end-to-end pipeline Eventsim $\rightarrow$ Kafka $\rightarrow$ Flink $\rightarrow$ Snowflake $\rightarrow$ dbt $\rightarrow$ Airflow $\rightarrow$ Power BI
5. Visualise key business metrics with **Power BI**

In Scope:

- Full pipeline from event simulation to BI
- Kafka ZooKeeper $\rightarrow$ KRaft evolution
- Spark micro-batch vs Flink true-streaming
- dbt star schema + Airflow orchestration
- Local Docker Compose deployment

Out of Scope:

- Real-time ML recommendations
- Multi-region K8s deployment
- Schema evolution (Avro/CDC)

Table of Contents

Why Apache Kafka?

Problems with direct service communication:

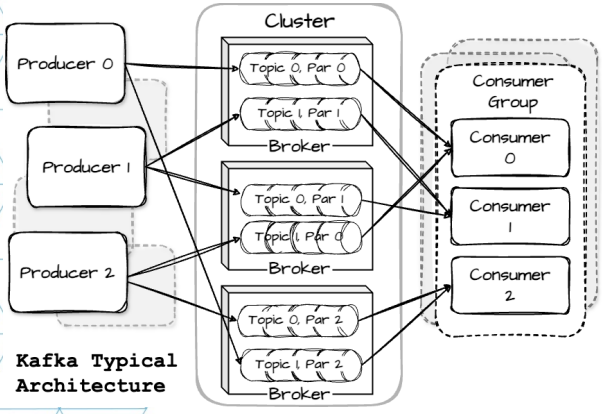
- **Tight coupling** — producers must know every consumer's address & schema
- **No buffering** — slow consumer blocks or drops messages
- **No replay** — message consumed = gone; no recovery on failure
- **Fan-out** — N consumers = N separate calls per producer

How Kafka solves it:

- **Decoupling** — producers write to topics; consumers subscribe independently
- **Durable buffering** — records persisted to disk & replicated
- **Replay** — configurable retention; re-read past events anytime
- **Horizontal scalability** — partitions linearly scale throughput
- **Fan-out for free** — multiple consumer groups, each own offset

viettel
Theo cách của bạn

Kafka Architecture



- **Producer** — partitions by key hash, batches, compresses; `acks=all` for durability
- **Broker** — leader/follower replicas; append-only segment log; sequential I/O
- **Topic** — named channel; immutable records; time-based or log-compaction retention
- **Partition** — ordered, offset-indexed; unit of parallelism
- **Consumer Group** — each partition → exactly one member; independent offset cursor per group

viettel
Theo cách của bạn

Table of Contents

Batch vs. Stream Processing

Batch Processing:

- Collect data over a window → store → process in bulk
- Latency: **hours to days**
- Tools: Hadoop MapReduce, scheduled SQL
- Good for: historical reports, large-scale transforms

Stream Processing:

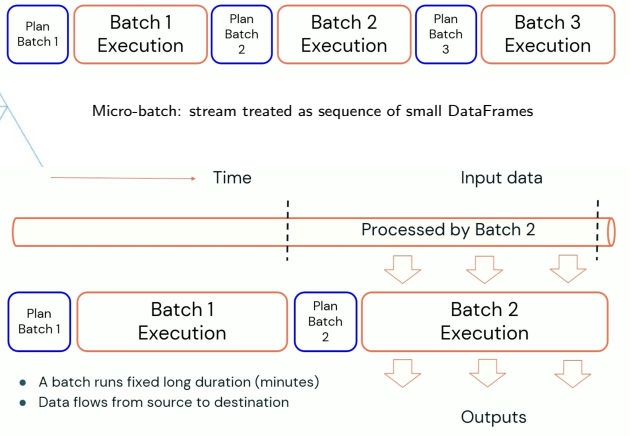
- Computation runs *continuously* as records arrive
- Latency: **milliseconds to seconds**
- Tools: Apache Flink, Spark Structured Streaming

Key Challenges in Stream Processing:

1. **Unbounded data** — no defined end; cannot accumulate unbounded state
2. **Out-of-order events** — network delays, clock skew; need watermarks
3. **Stateful computation** — aggregations & joins require fault-tolerant keyed state
4. **Fault tolerance** — any node can fail; must resume without data loss or duplicates
5. **Delivery semantics** — at-most-once, at-least-once, exactly-once

viettel
Theo cách của bạn

Apache Spark Structured Streaming



Micro-batch (default):

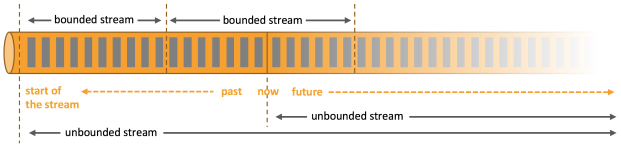
- Trigger interval configurable (e.g. 1 s)
- WAL + idempotent sinks for fault tolerance
- Latency: ∼**100ms–seconds**
- Exactly-once with idempotent sinks

Continuous (RTM):

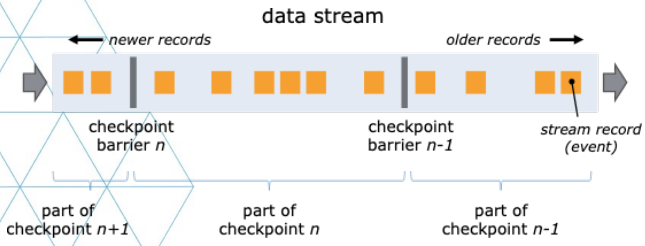
- Record-at-a-time; epoch checkpointing
- Latency: ∼**1ms**
- At-least-once semantics

Unified API: same code runs in batch and streaming mode

Apache Flink: True Event Streaming



True streaming dataflow: operators process records one-by-one



Async barrier snapshotting: barriers flow through the DAG triggering consistent snapshots

Dataflow model:

- Records processed **one-by-one** — no micro-batching
- Operators form a DAG; data flows continuously
- **Event time** + watermarks handle out-of-order events

Fault tolerance:

- Barrier checkpoints; state in RocksDB
- Recovery from last complete checkpoint
- **Exactly-once** with transactional sinks

APIs: DataStream, Table API, Flink SQL, **PyFlink**

viettel
Theo cách của bạn

Spark vs. Flink: Side-by-Side

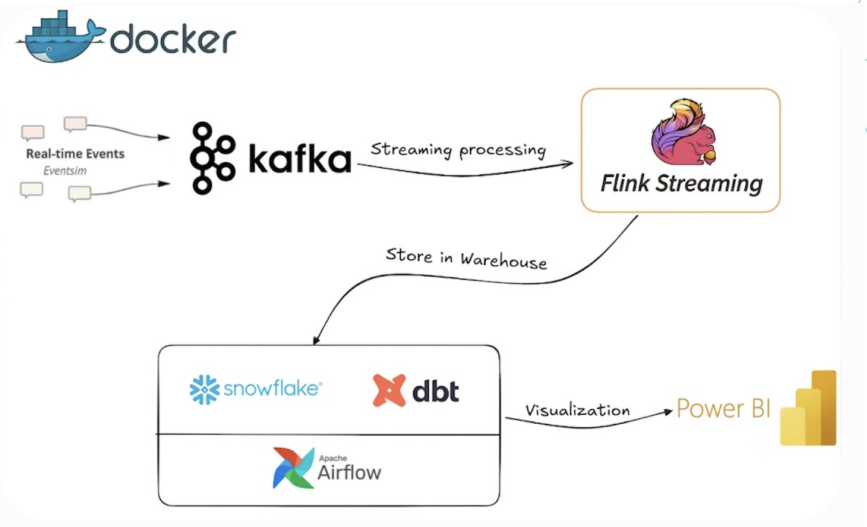
Dimension	Spark Structured Streaming	Apache Flink
Execution model	Micro-batch (default); Continuous (RTM)	True event-by-event streaming
Latency	∼100ms–seconds (micro-batch); ∼1ms (RTM)	∼1–10ms
Fault tolerance	WAL + idempotent sinks	Async barrier snapshotting
State backend	In-memory / HDFS	RocksDB (incremental) / heap
Exactly-once	Yes (micro-batch + idempotent sink)	Yes (transactional sink)
Watermarks	Watermark delay column	Native event-time watermarks
Python support	PySpark (mature)	PyFlink (growing)
Use case sweet spot	Batch + streaming unified; ML pipelines	Low-latency CEP, stateful pipelines

Why Flink for Streamify: lower native latency, richer event-time semantics, and

StatementSet pattern allows 2 Kafka→Snowflake pipelines in **one job** with one shared

Table of Contents

Streamify: End-to-End Architecture



Streaming Pipeline: Eventsim → Kafka → Flink → Snowflake

Event Simulation (Eventsim):

- 10,000-song subset of Million Songs Dataset
- Synthetic user demographics (US census data)
- Publishes 3 topics simultaneously with Schema Registry enforcement

PyFlink Job (`stream_events.py`):

- `create_kafka_source()` registers DDL source table per topic
- Timestamp: `TO_TIMESTAMP_LTZ(ts/1000, 3)`
- Partition columns: `year_`, `month_`, `day_`, `hour_`
- `string_decode` UDF: latin1 → UTF-8 for artist/song names

Kafka Topics:

- `listen_events` — song play metadata
- `page_view_events` — navigation & HTTP context
- `auth_events` — session & subscription data

Snowflake Sink:

- JDBC connector with exactly-once guarantees
- Raw events land in `STAGING` schema
- `processed_time` column tracks pipeline latency

One checkpoint cycle for all 3 pipelines — shared fault recovery

Data Modeling & Orchestration

dbt Star Schema (inside Snowflake):

- `fact_streams` — one row per song play; FK to all 5 dims
- `dim_users` — demographics & subscription level
- `dim_songs` — title, duration, artist reference
- `dim_artists` — name & location
- `dim_locations` — city, state, coordinates
- `dim_datetime` — year/month/day/hour/weekday hierarchy

Wide Table (`wide_streams`):

- Pre-joined denormalised view of all 6 tables
- Power BI users query **one flat table** — no DAX joins needed

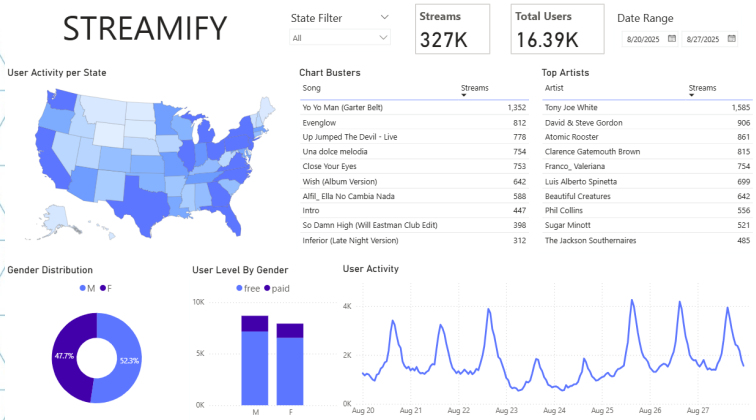
Apache Airflow Orchestration:

- DAG: `airflow/dags/dbt.py`
- Schedule: **daily**
- Task: `DockerOperator` runs `dbt run` in `dbt-snowflake` container
- Docker isolation keeps Airflow worker clean; dbt version pinned
- Container auto-removed on success

ELT Pattern:

Flink **loads** raw data → dbt **transforms** inside Snowflake.
Streaming concerns cleanly separated from analytical modelling.

Power BI Dashboard



Key metrics:

- Top artists by play count
- Songs played over time (hourly)
- Geographic distribution (US map)
- Free vs. paid subscription tiers
- Session activity & auth rates

Table of Contents

Summary: Key Takeaways

1. **Apache Kafka** provides reliable, replay-capable event transport. Producers and consumers are fully decoupled — the processing engine (Spark or Flink) can be **swapped without touching the pipeline's upstream or downstream**.

2. **Apache Flink**'s Table API and `StatementSet` pattern allow 3 independent Kafka→Snowflake pipelines to run as a **single job** sharing one checkpoint cycle and one set of TaskManager slots.

3. The **ELT pattern** cleanly separates concerns:
   - Flink handles **real-time loading** of raw events
   - dbt handles **analytical transformation** inside Snowflake
   - Airflow handles **orchestration and scheduling**

4. **Flink outperforms Spark** for low-latency, stateful streaming workloads — native event-time semantics, incremental RocksDB state, and millisecond-level latency with exactly-once guarantees.

viettel
Theo cách của bạn

Future Developments

Near-term

- **Flink CDC** — replace Eventsim with Debezium CDC connector to stream changes from a real operational database
- **Schema evolution** — integrate Confluent Schema Registry with Avro serialisation for backward-compatible changes

Long-term

- **Real-time ML** — embed a Flink ML model for song recommendations; write results to Redis serving store
- **Cloud deployment** — migrate from Docker Compose to Amazon Kinesis Data Analytics (Flink) + Confluent Cloud (Kafka)

- THE END -

Thank you for your attention

Contact:
Ngo Quang Hieu — Viettel Digital Talent 2026